

REGISTRATION & LOGIN

الرحلة الكاملة للمؤسسة والمستخدم والمصادقة

من أول Register Tenant حتى Access Token و Protected Routes

هدف الدليل

شرح كل ملف أنشأناه في مرحلة Registration و Login، مسؤولية كل طبقة، حركة البيانات، ال Transactions، كلمات المرور، JWT، وحماية ال APIs.

2026 أغسطس - Legal RAG App

1. خريطة المرحلة كاملة

```
01 Registration / Login Request
02   -> Pydantic Schemas
03   -> Auth Route
04   -> AuthController
05   -> RegistrationService or Logic Models
06   -> PasswordHasher + JWTService
07   -> PostgreSQL
08   -> Access Token + Refresh Token
09
10 Protected Request
11   -> HTTPBearer Dependency
12   -> Current User
13   -> Tenant-aware Route
```

المسؤولية	الطبقة
فحص وتنظيف بيانات التسجيل والدخول وتحديد شكل الاستجابات.	Schemas
Hash كلمات المرور وإنشاء وفحص JWT.	Core
إنشاء Tenant و Admin و Roles داخل Transaction واحدة.	RegistrationService
تنسيق Register و Login و Refresh و Current User.	AuthController
قراءة Access Token وحماية Routes وفرض الصلاحيات.	Dependencies
واجهة HTTP التي يستخدمها Frontend أو Swagger.	Routes

2. Registration Schema

الملف `src/schemas/registration.py` يحتوي `TenantRegistrationRequest`، وهو بوابة التحقق من بيانات صاحب المؤسسة قبل تشغيل أي منطق في قاعدة البيانات.

```
01 {
02   "tenant_name": "Kilani Legal Office",
03   "tenant_code": "kilani-legal",
04   "admin_full_name": "Youssef Kilani",
05   "admin_email": "yk787@gmail.com",
06   "password": "StrongPassword123",
07   "confirm_password": "StrongPassword123"
08 }
```

المعنى والتحقق	الحقل
الاسم الظاهر للمؤسسة، بطول محدد وبعد إزالة المسافات الزائدة.	tenant_name

tenant_code	كود فريد للمؤسسة، lowercase، حروف وأرقام وشرطة مفردة فقط.
admin_full_name	اسم أول مدير للمؤسسة.
admin_email	Gmail صحيحة فقط، وتتحول إلى lowercase.
password	8 أحرف على الأقل، حرف كبير، حرف صغير، ورقم.
confirm_password	يجب أن تطابق password تمامًا.

tenant_code

يختارها صاحب المؤسسة أو يقترحها النظام من اسم المؤسسة. تحفظ في جدول tenants وتستخدم في كل Login مستقبلية مع البريد وكلمة المرور.

Auth Schemas 3.

الملف src/schemas/auth.py يحدد شكل Login و Refresh و Token Response والمستخدم الحالي.

LoginRequest 3.1

```
01 {
02   "tenant_code": "kilani-legal",
03   "email": "yk787@gmail.com",
04   "password": "StrongPassword123"
05 }
```

- يحوّل tenant_code والبريد إلى lowercase.
- يفحص صيغة البريد عبر EmailStr.
- لا يتحقق من صحة كلمة المرور، لأن ذلك يحدث باستخدام PasswordHasher.

TokenResponse 3.2

```
01 {
02   "access_token": "eyJ...",
03   "refresh_token": "eyJ...",
04   "token_type": "bearer",
05   "expires_in": 1800
06 }
```

CurrentUserResponse 3.3

يرجع User و Tenant و Roles بدون password أو password_hash.

4. User Schemas

الملف `src/schemas/user.py` يستخدم لاحقًا عندما ينشئ Tenant Admin موظفين داخل المؤسسة أو يعدّل بياناتهم.

Schema	الاستخدام
UserCreate	إنشاء مستخدم جديد داخل Tenant مع Gmail وكلمة مرور.
UserUpdate	تعديل البريد أو الاسم أو الحالة أو الصفة الإدارية.
UserPasswordUpdate	تغيير كلمة المرور مع مطابقة التأكيد واختلافها عن القديمة.
UserResponse	إرجاع بيانات المستخدم الآمنة فقط.
UserListResponse	إرجاع قائمة المستخدمين مع Pagination.

5. PasswordHasher

الملف `src/core/password_hasher.py` مسؤول عن Hash وVerify فقط. لا يعرف شيئًا عن HTTP أو قاعدة البيانات.

```
01 password_hash = password_hasher.hash_password(  
02     plain_password  
03 )  
04  
05 is_valid = password_hasher.verify_password(  
06     plain_password,  
07     stored_password_hash,  
08 )
```

- نستخدم Argon2 لتخزين Password Hash آمنة.
- لا نخزن كلمة المرور الأصلية داخل PostgreSQL.
- وقت Login لا نفك ال Hash، بل نتحقق من التطابق.

تدفق كلمة المرور

Plain Password -> PasswordHasher -> Argon2 Hash -> users.password_hash

6. JWTService

الملف `src/core/jwt_service.py` مسؤول عن إنشاء وفك Access Token وRefresh Token والتحقق من التوقيع والصلاحيات ونوع التوكن.

Claim	المعنى
-------	--------

sub	معرف المستخدم user_id.
tenant_id	معرف المؤسسة التي ينتمي إليها المستخدم.
token_type	access أو refresh.
is_tenant_admin	هل المستخدم مدير المؤسسة.
roles	أسماء أدوار المستخدم داخل Access Token.
iat	وقت إصدار التوكن.
exp	وقت انتهاء الصلاحية.
jti	معرف فريد للتوكن.

6.1 الفرق بين التوكنز

Access Token	Refresh Token
تستخدم مع APIs المحمية.	تستخدم فقط لتجديد التوكنز.
عمرها قصير، مثل 30 دقيقة.	عمرها أطول، مثل 7 أيام.
تحتوي Roles وصلاحيه Admin.	لا تحتاج Roles، لأنها تحمل من DB عند التجديد.

7. RegistrationService

الملف src/services/registration_service.py ينفذ التسجيل الفعلي. أنشأناه لأن Register Tenant تحتاج عدة عمليات مترابطة يجب أن تنجح أو تفشل معًا.

```

01 BEGIN TRANSACTION
02     Create Tenant
03     Hash Password
04     Create Admin User
05     Create System Roles
06     Assign Tenant Admin Role
07 COMMIT
08
09 On any failure:
10     ROLLBACK

```

7.1 خطوات register_tenant()

- فحص أن tenant_code غير مستخدمة.
- إنشاء Tenant بحالة active وإعدادات فارغة.
- تحويل كلمة المرور إلى Argon2 Hash.
- إنشاء أول User مع is_tenant_admin = true.
- إنشاء Tenant Admin و Document Manager و Viewer.
- ربط أول Admin بدور Tenant Admin داخل user_roles.
- إعادة RegistrationResult إلى AuthController.

أهمية Transaction

إذا فشل إنشاء Role مثلاً، يتم إلغاء Tenant و User أيضاً، فلا تظل بيانات ناقصة داخل النظام.

8. AuthController

الملف src/controllers/AuthController.py هو المنسق الرئيسي. يستخدم الخدمات وال Logic Models دون أن يكرر مسؤولياتها.

الدالة	ما تفعله
register_tenant()	تستدعي RegistrationService ثم تصدر Token Pair لأول Admin.
login()	تبحث عن المستخدم، تفحص Password و User و Tenant، تحمل Roles، وتصدر Tokens.
refresh_tokens()	تفحص Refresh Token، تعيد تحميل User و Roles، ثم تصدر Tokens جديدة.
get_current_user()	تفك Access Token وتعيد User و Tenant و Roles من قاعدة البيانات.

8.1 البحث في Login

```
01 user = await user_model.get_user_by_tenant_code_and_email(  
02     tenant_code=login_data.tenant_code,  
03     user_email=login_data.email,  
04     include_roles=True,  
05 )
```

نستخدم tenant_code + email لأن البريد نفسه قد يكون موجوداً داخل أكثر من Tenant، وكل حساب يظل مستقلاً.

Auth Routes .9

الملف src/routes/auth.py يوفر واجهة HTTP فقط. لا ينفذ SQL أو Hash أو JWT بنفسه.

Endpoint	الحماية	النتيجة
POST /auth/register-tenant	عام	إنشاء المؤسسة وأول Admin ثم 201 مع Tokens.
POST /auth/login	عام	التحقق من البيانات ثم 200 مع Tokens.
POST /auth/refresh	Refresh Token	إصدار Token Pair جديدة.
GET /auth/me	Access Token	إرجاع User و Tenant و Roles.

تكرار tenant_code يرجع 409، بيانات Pydantic الخاطئة ترجع 422، وبيانات Login الخاطئة ترجع 401.

Auth Dependencies .10

الملف src/dependencies/auth.py يعمل قبل Routes المحمية. استخدمنا HTTPBearer لقراءة Authorization Header.

```
01 Authorization: Bearer ACCESS_TOKEN
02     -> get_current_user
03     -> validate JWT
04     -> load active User and Tenant
05     -> load current Roles
06     -> allow protected Route
```

Dependency	وظيفتها
get_current_user	إثبات الهوية وإرجاع CurrentUserResponse.
require_tenant_admin	السماح فقط لمدير المؤسسة.
("...")require_role	السماح فقط لمن يمتلك Role محددة، مع تجاوز Admin حسب السياسة الحالية.

401 مقابل 403
401 تعني أن الهوية غير مؤكدة. 403 تعني أن المستخدم معروف، لكن لا يمتلك الصلاحية المطلوبة.

11. رحلة Register Tenant كاملة

```
01 POST /api/v1/auth/register-tenant
02     -> TenantRegistrationRequest
03     -> validate tenant code, Gmail, and password
04     -> Auth Route
05     -> AuthController.register_tenant()
06     -> RegistrationService
07     -> BEGIN TRANSACTION
08     -> create Tenant
09     -> hash password
10     -> create first Admin User
11     -> create System Roles
12     -> assign Tenant Admin Role
13     -> COMMIT
14     -> JWTService.create_token_pair()
15     -> 201 Created
```

بعد نجاح التسجيل يدخل صاحب المؤسسة مباشرة، لأن Response تحتوي Access و Refresh Tokens.

12. رحلة Login كاملة

```
01 POST /api/v1/auth/login
02     -> LoginRequest
03     -> AuthController.login()
04     -> lookup by tenant_code + email
05     -> verify active User
06     -> verify password hash
07     -> verify active Tenant
08     -> load Roles
09     -> update last_login_at
10     -> create Access + Refresh Tokens
11     -> 200 OK
```

نستخدم رسالة invalid credentials سواء الحساب غير موجود أو كلمة المرور خاطئة، لتقليل كشف معلومات الحسابات.

13. رحلة استخدام API محمية

```
01 Frontend sends Access Token
02     -> Authorization: Bearer TOKEN
03     -> get_current_user dependency
04     -> JWTService decodes token
05     -> AuthController loads User, Tenant, Roles
06     -> Route gets current_user
07     -> tenant_id = current_user.user.tenant_id
08     -> Logic Model applies tenant isolation
```

Tenant Isolation

لا نأخذ tenant_id من Request Body. نأخذها من current_user بعد التحقق، ثم نمررها إلى كل Query حساسة.

14. الملفات النهائية ومسؤولياتها

المسؤولية	الملف
فحص تسجيل المؤسسة وأول Admin.	schemas/registration.py
Current User و Token Payloads و Refresh و Login	schemas/auth.py
إنشاء وتعديل وإرجاع المستخدمين.	schemas/user.py
Hash و Verify لكلمات المرور.	core/password_hasher.py
إنشاء وفحص Access و Refresh Tokens.	core/jwt_service.py
Transaction إنشاء Tenant و Admin و Roles.	services/registration_service.py
تنسيق Register و Login و Refresh و Current User.	controllers/AuthController.py
حماية Routes وفرض Admin أو Role.	dependencies/auth.py
Endpoints التي يستخدمها Frontend.	routes/auth.py

15. حالات الاختبار الأساسية

النتيجة المتوقعة	الاختبار
201 مع Access و Refresh Tokens.	Register ببيانات صحيحة
409 Conflict.	Register بنفس tenant_code
422 Validation Error.	Register ببيانات غير صحيحة
200 مع Token Pair.	Login صحيح
401 Unauthorized.	Login بكلمة مرور خاطئة
401 Unauthorized.	me/ بدون Access Token
200 مع User و Tenant و Roles.	me/ بتوكن صحيحة
200 مع Token Pair جديدة.	Refresh صحيحة

16. الخلاصة

النتيجة

أنشأنا مسارًا كاملاً وآمناً يبدأ بتسجيل مؤسسة وأول Admin داخل Transaction، ثم Login وإصدار Tokens، ثم حماية بقية النظام عبر Dependencies و Tenant Isolation.

المرحلة التالية: **Users Routes** ثم **Roles Routes** ثم **UserRole Routes**، حتى يستطيع **Tenant Admin** إنشاء الموظفين وإدارة صلاحياتهم.